

Desenvolvimento Ágil - Extreme Programming (XP)

Profa. Rainara Maia Carvalho

XP não define um passo a passo detalhado para o desenvolvimento de software

Valores, Princípios e Práticas

Valores

comunicação, simplicidade, feedback,
coragem, respeito e qualidade de vida.

Comunicação

Uma boa comunicação é importante em qualquer projeto, não apenas para evitar, mas também para aprender com erros



Simplicidade

Em todo sistema complexo e desafiador existem sistemas ou subsistemas mais simples, que às vezes não são considerados



Feedback

Um valor que ajuda a controlar tais riscos é estar aberto ao feedback dos stakeholders, a fim de que correções de rota sejam implementadas o quanto antes. Em outras palavras, é difícil desenvolver o sistema de software certo em uma primeira e única tentativa.

"Planeje-se para jogar fora partes do seu sistema, pois você fará isso."

Frederick Brooks

Feedback é um valor essencial para garantir que as partes ou versões que serão descartadas sejam identificadas o quanto antes, de forma a diminuir prejuízos e retrabalho.



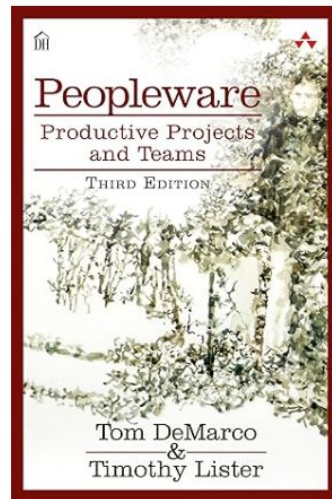
Princípios

ponte entre os valores e as
práticas

Humanidade

Software é uma atividade intensiva no uso de capital humano. O **principal recurso** de uma empresa de software não são seus bens físicos — computadores, prédios, móveis ou conexões de Internet, por exemplo — mas sim seus **colaboradores**.

Termo "peopleware"



Economicidade

Software é uma atividade cara, que demanda a alocação de recursos financeiros consideráveis.

Logo, tem-se que ter consciência de que o outro lado, isto é, quem está pagando as contas do projeto, espera resultados econômicos e financeiros.

Por isso, na grande maioria dos casos, software não pode ser desenvolvido apenas para satisfazer a vaidade intelectual de seus desenvolvedores.

Software não é uma obra de arte, mas algo que tem que gerar resultados econômicos, como defendido por esse princípio de XP.



Benefícios Mútuos

XP defende que as decisões tomadas em um projeto de software têm que beneficiar múltiplos stakeholders.

Exemplo: testes, refatoração



Melhorias Contínuas

Nenhum processo de desenvolvimento de software é perfeito. Por isso, é mais seguro trabalhar com um sistema que vai sendo continuamente aprimorado, a cada iteração, com o feedback dos clientes e de todos os membros do time



Falhas acontecem

Desenvolvimento de software não é uma atividade livre de riscos.

Software é uma das mais complexas construções humanas. Logo, falhas são esperadas em projetos de desenvolvimento de software.

Evidentemente, XP não advoga que essas falhas devem ser acobertadas. Porém, elas não devem ser usadas para punir membros de um time. Pelo contrário, falhas fazem parte do jogo, se um time pretende entregar software com rapidez.



Baby Steps

É melhor um progresso seguro, testado e validado, mesmo que pequeno, do que grandes implementações com riscos de serem descartadas pelos usuários.

Exemplo: testes, integração de código e refatorações

Em resumo, o importante é garantir melhorias contínuas, não importando que sejam pequenas, desde que na direção correta. Essas pequenas melhorias são melhores do que grandes revoluções, as quais costumam não apresentar resultados positivos, pelo menos quando se trata de desenvolvimento de software.



Responsabilidade Social

De acordo com esse princípio, desenvolvedores devem ter uma ideia clara de seu papel e responsabilidade na equipe. O motivo é que responsabilidade não pode ser transferida, sem que a outra parte a aceite.

Por isso, XP defende que o engenheiro de software que implementa uma história — termo que o método usa para requisitos — deve ser também aquele que vai testá-la e mantê-la.



Práticas

Processo, Programação e Gerência

Práticas sobre o Processo de Desenvolvimento

Representante dos clientes

Histórias de Usuários, Story Points
(Planning Poker)

Iterações, Releases, Velocidade

Planejamento, Slacks (folgas)

Representante do Cliente e Histórias de Usuário

Os times incluem pelo menos um representante dos clientes, que deve entender do domínio do sistema que será construído.

Uma das funções desse representante é escrever as histórias de usuário (que é um tipo de requisito)

Exemplo de História de Usuário do Stack Overflow

Postar Pergunta

Um usuário, quando logado no sistema, deve ser capaz de postar perguntas. Como é um site sobre programação, as perguntas podem incluir blocos de código, os quais devem ser apresentados com um layout diferenciado.

Estimativa - Story Points

Os desenvolvedores definem quanto esforço será necessário para implementar as histórias escritas pelo representante dos clientes.

Frequentemente, a duração de uma história é estimada em **story points**, em vez de horas ou homens/hora.

Usa-se uma escala inteira para classificar histórias como possuindo um certo número de story points.

O objetivo é **definir uma ordem relativa entre as histórias**. As histórias mais simples são estimadas com 1 story point; histórias que são cerca de duas vezes mais complexas do que as primeiras são estimadas com 2 story points e assim por diante...

Sequência Fibonacci 1, 2, 3, 5, 8, 13

Por que story points e não horas?

O propósito é diferente: horas medem tempo (“quanto vai demorar”). Story points medem esforço relativo — uma combinação de: complexidade técnica, volume de trabalho, incerteza ou risco

Estimar em horas tende a focar em quanto tempo individualmente vou levar, enquanto story points focam em quanto esforço o time precisa aplicar em relação a outras histórias.



Por que story points e não horas?

Story points permitem comparação relativa: se uma história A é 2 pontos e uma história B parece o dobro de complexa, B vira 4 pontos. Isso cria consistência interna no time — mesmo que as horas variem entre pessoas, o esforço relativo se mantém.

Já as horas variam muito de pessoa pra pessoa: um dev sênior pode levar 2h, um júnior 6h, mas o esforço percebido é o mesmo.



Por que story points e não horas?

Estimar em horas cria falsa precisão: quando o time diz “isso leva 5 horas”, a tendência é tratar essa previsão como um compromisso rígido, não como uma estimativa. Story points, por serem uma medida abstrata, reduzem essa ilusão de precisão e ajudam o time a falar sobre incerteza.



Por que story points e não horas?

Story points favorecem a colaboração do time: durante a estimativa, todos discutem juntos o tamanho da história. Isso: estimula compartilhamento de conhecimento, melhora o entendimento coletivo, alinha percepções sobre complexidade.

Com horas, tende a virar uma conversa tipo “quanto tempo você leva?”, o que individualiza e desmotiva o processo.



Planning Poker

técnica usada para estimar o tamanho de histórias

O representante dos clientes seleciona uma história e a lê para os desenvolvedores.

Os desenvolvedores interagem com o representante dos clientes para tirar possíveis dúvidas e conhecer melhor a história.

Cada desenvolvedor faz sua estimativa para o tamanho da história, de forma independente.

Eles ao mesmo tempo levantam cartões com a estimativa que pensaram, em story points.

Se houver consenso, o tamanho da história está estimado e passa-se para a próxima história.

Senão, o time deve iniciar uma discussão, para esclarecer a razão das diferentes estimativas.

Feito isso, realiza-se uma nova votação e o processo se repete, até que o consenso seja alcançado.

Dinâmica - Planning Poker

Iterações, Releases e Velocidade

A implementação das histórias ocorre em **iterações**, as quais têm uma duração fixa e bem definida, variando de **uma a três semanas**, por exemplo.

As iterações, por sua vez, formam ciclos mais longos, chamados de **releases**, de dois a três meses, por exemplo.

A **velocidade** de um time é o número de story points que ele consegue implementar em uma iteração.



Em resumo, para começar a usar XP precisamos:

- Definir a duração de uma iteração.
- Definir o número de iterações de uma release.
- Um conjunto de histórias, escritas pelo representante dos clientes.
- Estimativas para cada história, feitas pelos desenvolvedores.
- Definir a velocidade do time, isto é, o número de story points que ele consegue implementar por iteração.



Planejamento da Release

O representante do cliente deve **priorizar as histórias**.

Para isso, ele deve **definir quais histórias serão implementadas nas iterações da primeira release**.

Nessa priorização, deve-se respeitar a velocidade do time de desenvolvimento.



Exemplo - Stack Overflow - Qual a velocidade?

História de Usuário	Story Points	Iteração	Release
Cadastrar usuário	8	1	1
Postar perguntas	5	1	1
Postar respostas	3	1	1
Tela de abertura	5	1	1
Gamificar perguntas/respostas	5	2	1
Pesquisar perguntas/respostas	8	2	1
Adicionar tags	5	2	1
Comentar perguntas/respostas	3	2	1

Planejamento da iteração

Uma vez realizado o planejamento de uma release, começam as iterações.

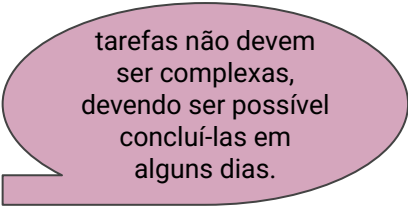
Antes de mais nada, o time de desenvolvimento deve se reunir para realizar o planejamento da iteração.

O objetivo desse planejamento é **decompor as histórias de uma iteração em tarefas**, as quais devem corresponder a atividades de programação que possam ser alocadas para um dos desenvolvedores do time.



Exemplo História Postar Perguntas

- Projetar e testar a interface Web, incluindo layout, CSS templates, etc.
- Instalar banco de dados, projetar e criar tabelas.
- Implementar a camada de acesso a dados.
- Instalar servidor e testar framework web.
- Implementar camada de controle, com operações para cadastrar, remover e atualizar perguntas.
- Implementar interface Web.



tarefas não devem
ser complexas,
devendo ser possível
concluí-las em
alguns dias.

Folgas

XP defende ainda que os times, durante uma iteração, programem algumas folgas (**slacks**), que são tarefas que podem ser adiadas, caso necessário.

Ex: estudo de uma nova tecnologia, a realização de um curso online, preparar uma documentação ou manual ou mesmo desenvolver um projeto paralelo.

Dois objetivos:

- (1) criar um buffer de segurança em uma iteração, que possa ser usado caso alguma tarefa demande mais tempo do que o previsto;
- (2) permitir que os desenvolvedores respirem um pouco, pois o ritmo de trabalho em projetos de desenvolvimento de software costuma ser intenso e desgastante.

Dinâmica - Velocidade do time XP

Práticas sobre Gerenciamento de Projetos

Ambiente de Trabalho
Contratos com Escopo Aberto
Métricas de processo

Ambiente de Trabalho

Times pequenos

Trabalho em uma mesma sala, para facilitar comunicação e feedback.

Espaço de trabalho seja informativo

Garantia de jornadas de trabalho sustentáveis



Contrato de Escopo Aberto x Fechado

Escopo fechado

- Requisitos são definidos antecipadamente pela contratante.
- A contratada define preço e prazo de entrega.
- Problemas comuns:
 - Requisitos mudam durante o projeto.
 - Cliente nem sempre sabe exatamente o que quer.
 - Pode levar a entregas com baixa qualidade, requisitos incompletos ou com bugs.
 - Empresas podem priorizar evitar multas ao invés de garantir qualidade.

Contrato de Escopo Aberto x Fechado

Escopo aberto

- Pagamento é feito por hora trabalhada.
- A contratada aloca um time de desenvolvedores para o projeto.
- Define-se o valor da hora de cada membro da equipe.
- Cliente cria histórias e valida ao final de cada iteração.
- Contrato pode ser renovado ou encerrado após alguns meses.
- Dá liberdade ao cliente de trocar de empresa se não estiver satisfeito.



Métricas de Processo

Para que gerentes e executivos possam acompanhar um projeto XP recomenda-se o uso de duas métricas principais:

- número de bugs em produção (que deve ser idealmente da ordem de poucos bugs por ano); e
- intervalo de tempo entre o início do desenvolvimento e o momento em que o projeto começar a gerar os seus primeiros resultados financeiros (que também deve ser pequeno, da ordem de um ano, por exemplo).



Práticas sobre o Programação

Design Incremental
Programação em Pares
Propriedade Coletiva do Código
Testes Automatizados
Desenvolvimento Dirigido por
Testes
Build Automatizado
Integração Contínua

"O nome Extreme Programming foi escolhido porque XP propõe um conjunto de práticas de programação inovadoras, principalmente para a época na qual foram propostas, no final da década de 90. "

Design Incremental

XP evita design detalhado inicial; prefere design incremental durante todo o projeto.

Simplicidade e adaptação: requisitos mudam, surgem novos, e design inicial pode ficar obsoleto.

Vantagem: design contínuo reduz riscos, alinha-se às necessidades reais e aumenta eficiência.

Design incremental somente é possível caso seja adotado em conjunto com as demais práticas de XP, principalmente **refactoring**

Propriedade Coletiva do Código

Qualquer desenvolvedor pode modificar qualquer parte do código:

Implementar novas funcionalidades, Corrigir bugs, Aplicar refactoring.

Exemplo: Se um bug é encontrado, qualquer desenvolvedor pode corrigi-lo imediatamente.

Sem necessidade de autorização.

Testes Automatizados

Testes manuais é um procedimento custoso e que não pode ser reproduzido a todo momento.

XP propõe a implementação de programas — chamados de testes — para executar pequenas unidades de um sistema, como métodos, e verificar se as saídas produzidas são aquelas esperadas.

JUnit... XUnit...

Desenvolvimento Dirigido por Testes

Implementação do teste de um método antes do seu código.

TDD, que também é conhecido como test-first programming, possui duas motivações principais:

- (1) evitar que a escrita de testes seja sempre deixada para amanhã, pois eles são a primeira coisa que se deve implementar;
- (2) ao escrever um teste, o desenvolvedor se coloca no papel de cliente do método testado

Build Automatizado

Geração de uma versão de um sistema que seja executável e que possa ser **colocada em produção**.

Inclui não apenas a compilação do **código**, mas a execução de outras **ferramentas** como linkeditores e empacotadores de código em arquivos WAR, JAR, etc.

XP defende que o processo de build seja **automatizado**, sem nenhuma intervenção dos desenvolvedores.

XP defende que o processo de build seja o **mais rápido possível**, para que os desenvolvedores recebam rapidamente feedback sobre possíveis problemas, como um erro de compilação.

Integração Contínua

Sistemas de Controle de Versão (ex. Git)

Objetivo da IC: evitar que desenvolvedores passem muito tempo trabalhando localmente, em suas tarefas, sem integrar o código. E, com isso, pelo menos diminuir as chances e o tamanho dos conflitos.

Serviço de integração contínua: antes de realizar qualquer integração, esse serviço faz o build do código e executa os testes. O objetivo é garantir que o código não possui erros de compilação e que ele passa em todos os testes.

Programação em Pares

Toda tarefa de **codificação** — incluindo implementação de uma nova história, de um teste, ou a correção de um bug — deve ser realizada por **dois desenvolvedores** trabalhando juntos, compartilhando o mesmo teclado e monitor.

Um dos desenvolvedores é o **líder** (ou driver) da sessão, ficando com o teclado e o mouse. Ao segundo desenvolvedor cabe a função de revisor e questionador (**navegador**).

Espera-se melhorar a qualidade do código e do design e disseminar o conhecimento sobre o código, que não fica nas mãos e na cabeça de apenas um desenvolvedor.

Desvantagens?

Dúvidas?

rainara@ufc.br

Cap. 2 - XP

